

THAW A CREATURE

Two-Developer Production Workflow — v0.2

Purpose: Define how the remaining *Thaw A Creature* work is split, coordinated, reviewed, and merged once a second engineer joins the project.

Current checkpoint: D7-R07 - Regional Cold Severity complete

Next implementation task: D7-R08 - Ancient Ice

Current progress: D7-R01 through D7-R07 are complete. The two-developer workflow now applies to the remaining Day-7 work and all later production.

Team size: 2 developers

Primary goal: Increase parallel throughput without creating conflicting implementations, design drift, or unstable shared systems.

This is an **operational production document**, not a new design authority. It does not supersede the canonical GDD, Technical Specifications, Constants, or Development Plan.

v0.2 Progress Update

- D7-R01 through D7-R07 are complete.
 - The Project Owner completed the Day-7 sequence through D7-R07 - Regional Cold Severity.
 - The next canonical task is D7-R08 - Ancient Ice.
 - The remaining Day-7 implementation is now D7-R08 through D7-R13.
 - The ownership plan has been re-baselined from the actual current checkpoint rather than the earlier post-D7-R03 assumption.
 - Week 2 and Week 3 role ownership remain unchanged unless the End-of-Week-1 gate produces a new canonical revision.
-

1. Canonical Authority Remains Unchanged

When implementation questions arise, continue using the established authority order:

1. 01_Thaw_A_Creature_Concise_GDD_Canonical_v0.5
2. 03_Thaw_A_Creature_Bare_Minimum_Prototype_Technical_v0.9 for Week-1 prototype behavior
3. 02_Thaw_A_Creature_Technical_Implementation_3Week_v0.5 for launch architecture

4. 05_Thaw_A_Creature_Implementation_Constants_Canonical_v0.9 for exact values and configuration
5. 04_Thaw_A_Creature_Development_Plan_Canonical_v0.9 for task order and gates
6. This workflow document for **ownership, coordination, branching, integration, and review**

If this workflow appears to conflict with a canonical design or technical document, the canonical document wins.

2. Team Roles

2.1 Design / Content Lead

Primary owner: Project owner

Owns:

- game-design decisions;
- player-facing intent;
- economy and progression tuning;
- creature content assignments;
- Ice pool/content mapping decisions;
- level/world layout;
- authored spawn placement;
- Hearth/world presentation;
- creature asset direction and approval;
- UX/UI intent;
- onboarding;
- Hero Moments;
- prototype and launch acceptance gates;
- qualitative playtesting;
- deciding whether a gameplay ambiguity requires a design change.

The Design / Content Lead may still implement code, especially presentation, UI, configuration, world, and content-facing tasks.

2.2 Systems Engineering Lead

Primary owner: Engineer

Owns:

- server-authoritative gameplay systems;
- persistent player data;
- schema migration;
- DataStore safety;
- multiplayer state;
- BaseService;
- ownership/isolation;
- remote validation and exploit protection;
- world-cycle synchronization;
- Special Ice lifecycle;
- service architecture;
- state invariants;
- technical integration;
- save/load and rejoin correctness;
- multiplayer QA;
- exploit QA;
- launch-blocking technical defects.

The Systems Engineering Lead must not independently make unresolved gameplay-design decisions.

2.3 Shared Responsibilities

Both developers share responsibility for:

- code review;
- Studio runtime testing;
- regression testing;
- keeping `main` playable;

- reporting touched files and behavior;
 - identifying dependency conflicts before starting work;
 - preserving the core retrieval loop;
 - protecting canonical scope;
 - removing temporary diagnostics before merge.
-

3. Non-Negotiable Collaboration Rules

Rule 1 — One Primary Owner Per Numbered Task

Every numbered task has exactly one primary implementer. The other developer may review, test, provide assets, provide design decisions, or fix a blocking defect with coordination. They should not independently implement a competing version of the same task.

Rule 2 — Do Not Parallel-Edit the Same Core Subsystem

Avoid simultaneous work in high-conflict systems such as:

- `PlayerStateService`
- `WarmthService`
- `IceService`
- `ThawService`
- `CreatureService`
- `WorldCycleService`
- `BaseService`
- `DataService`
- shared Remote dispatch/validation
- shared persistent schema
- shared config modules when both tasks change the same records

If two tasks require the same subsystem, sequence them.

Rule 3 — `main` Must Remain Playable

Before merge:

1. implementation is complete;
2. Studio runtime test passes;
3. regressions are checked;
4. temporary diagnostics are removed;
5. changed files are reviewed;
6. branch is synchronized with current `main`;
7. final sanity test passes.

Rule 4 — Do Not Invent Missing Gameplay

If a requirement is purely technical and behavior-preserving, choose the smallest robust implementation.

If it affects player-facing behavior or balance, stop and ask the Design / Content Lead.

Rule 5 — Design Revision Before Productionizing New Design

Prototype code does not automatically become launch design. The Day-5 Survival Pressure experiment remains bounded by the canonical documents. Do not expand combat/enemy scope merely because its prototype gate passed.

If a design change is approved:

1. revise the appropriate canonical design/technical documents;
2. update Constants if values/configuration changed;
3. update the Development Plan if sequencing/scope changed;
4. only then productionize the new behavior.

4. Standard Task Lifecycle

Use this process for every numbered implementation task.

Step 1 — Assign

Record:

- Task ID
- Primary owner
- Reviewer
- Dependencies
- Expected touched systems

Step 2 — Read the Source

The implementer reads:

1. exact Development Plan task;
2. relevant Technical Specification;
3. Constants;
4. GDD only when player-facing intent needs clarification.

Step 3 — Define Completion Before Coding

```
Task:
Owner:
Dependencies:
Expected files/systems:
Done when:
Runtime test:
Must not change:
```

Step 4 — Create a Task Branch

Recommended naming:

```
cm/D7-R04-hearth-prominence  
eng/D7-R07-regional-cold  
eng/L-D8-T01-dataservice  
cm/F-D15-T03-region-differentiation
```

Do not work directly on `main`.

Step 5 — Implement Only the Task

Do not automatically start the next task, add convenience systems, refactor unrelated systems, create generalized frameworks, or rewrite stable behavior unless required.

Step 6 — Self-Test in Studio

Test through actual gameplay/server lifecycle. Do not treat Command Bar `require()` as proof that live state works correctly.

Step 7 — Produce a Handoff Report

TASK:
OWNER:

FILES CREATED:
-

FILES MODIFIED:
-

BEHAVIOR IMPLEMENTED:
-

HOW TO TEST:
1.
2.
3.

EXPECTED SUCCESS:
-

KNOWN LIMITATIONS:
-

TEMPORARY DIAGNOSTICS:
None / list

READY FOR REVIEW:
Yes / No

Step 8 — Review

The other developer reviews for scope, architecture, regressions, accidental design changes, unsafe client authority, duplicate logic, and config values incorrectly hardcoded in services.

Step 9 — Final Runtime Sanity Test

Run the task's success path plus at least one nearby regression path.

Step 10 — Merge

Merge only after verified runtime. After merge, update `main`, confirm a clean tree, notify the other developer, and start the next dependent branch from updated `main`.

5. Git / Integration Strategy

5.1 Branch Model

```
main
├── cm/<task-id>-<short-name>
└── eng/<task-id>-<short-name>
```

`main` is the integration branch and should remain playable.

5.2 Merge Rule

Prefer:

finish -> test -> review -> merge -> next dependent task

rather than long-lived feature branches.

5.3 Shared-File Lock Rule

Before changing a high-conflict file, post:

LOCK:
Task: D7-R07
Files:
- WarmthService
- RegionConfig

Owner: Engineer
Expected unlock: after D7-R07 merge

The other developer avoids those files until merge unless both explicitly coordinate.

6. Current Split — Day 7 After D7-R07

The following Day-7 tasks are already complete:

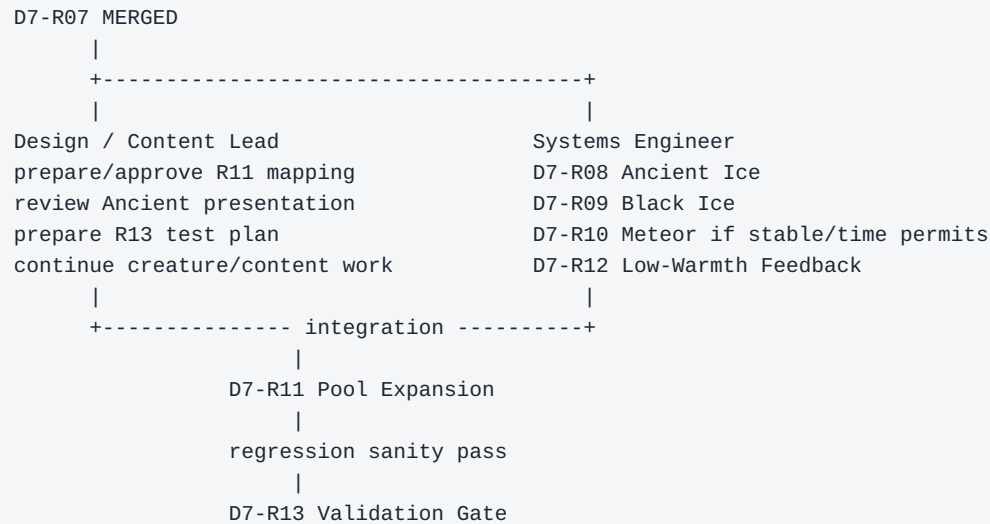
D7-R01 - Big-Number Warmth Rebalance
D7-R02 - Scalable Hearth Refill
D7-R03 - Hearth Progression Expansion
D7-R04 - Hearth Prominence Pass
D7-R05 - Expanded Progression Regions
D7-R06 - Recommended Warmth Signs
D7-R07 - Regional Cold Severity

The remaining Day-7 tasks are:

Task	Primary	Support / Review	Dependency Notes
D7-R08 - Ancient Ice	Systems Engineer	Design Lead provides/approves authored points, pool, presentation, and region intent	Next canonical task
D7-R09 - Black Ice	Systems Engineer	Design review	Only after R08 is stable
D7-R10 - Meteor Ice	Systems Engineer	Design review	Optional if time/stability permits; do not force it
D7-R11 - 11-Creature Pool Expansion	Design / Content Lead for explicit mapping; Engineer for integration support	Shared	Do not fabricate assignments

Task	Primary	Support / Review	Dependency Notes
D7-R12 - Low-Warmth Feedback	Systems Engineer for state/hooks; Design Lead for presentation acceptance	Shared	Percentage-based Safe -> Low -> Critical
D7-R13 - Progression Validation Gate	Design / Content Lead	Engineer participates	Final Day-7 / Week-1 validation

6.1 Recommended Parallel Sequence From the Current Checkpoint



Practical rule

The Design / Content Lead may prepare D7-R11 mappings while the engineer builds D7-R08 through D7-R10 , but final pool integration should use the actual stable Ice/config state that exists after those tasks. Do not pre-implement speculative pool structures.

D7-R12 can proceed as a separate systems task as long as it does not conflict with active Warmth/UI edits.

D7-R13 remains a hard validation point. Do not automatically begin launch persistence because one developer becomes free early.

7. End-of-Week-1 Gate

Before Week 2 begins, the team must explicitly decide whether the prototype has credible evidence for the core experience.

Verify:

- visible Ice creates desire;
- retrieval creates tension;
- near-failure creates retry desire;
- thawing creates anticipation;
- waiting has productive activity;
- Warmth training feels valuable;
- creature progression changes traversal;
- Great Frost refreshes interest;
- reveal creates another-Ice desire;
- later content creates aspiration.

Only after the gate is accepted should launch-production tasks begin.

8. Week 2 Ownership — Mechanical Launch Build

Day 8 — Persistence Foundation

Systems Engineer — Primary

- L-D8-T01 - DataService
- L-D8-T02 - Persistent Schema
- L-D8-T03 - Persistent Thaw Format
- L-D8-T04 - Load Failure Protection
- L-D8-T05 - Join Restore
- L-D8-T06 - Offline Thawing
- L-D8-T07 - Save / Rejoin Test

Design / Content Lead

- test real progression save/rejoin;
- deliberately test failure cases;
- verify player-facing restoration;
- continue remaining creature/content preparation;
- prepare content decisions for Days 11-12.

Day 9 — Multiplayer + Personal Bases

Systems Engineer — Primary

- server-size conversion;
- `BaseService` ;
- base ownership;
- shared wilderness;
- Ice contention;
- disconnect cleanup;
- remote validation;
- base isolation;
- private onboarding-Ice logic;
- depleted-field onboarding test.

Design / Content Lead

- base layout;
- shared-camp readability;
- `OnboardingIceSpawnPoint` placement;
- visual ownership readability;
- multiplayer gameplay validation;
- onboarding acceptance.

Day 10 — Production World Cycle

Systems Engineer — Primary

- wall-clock cycle;
- server synchronization;
- boot behavior;
- online-only Thaw Surge;
- production field reset;
- Special-Ice cadence;
- Special-Ice lifecycle;
- server-wide event hooks.

Design / Content Lead

- Special-Ice visual identity;
- Great Frost presentation;
- player-facing readability;
- reward-mystery acceptance.

Day 11 — 16 Creatures + Legendary

Design / Content Lead — Content Owner

- final roster decisions;
- remaining five launch creature assets;
- species presentation;
- silhouette mapping approval;
- content/rarity/pool approval;
- visual distinction.

Systems Engineer — Integration Owner

- Legendary technical/config support;
- all-16 ownership integration;
- Pen/Equip/Sell/Archive compatibility;

- Speed/stat integration;
- state consistency.

Day 12 — Five Ice Types

Design / Content Lead

- authored spawn placement;
- presentation differences;
- explicit pool mapping;
- rarity-weight design approval;
- long-distance mystery/readability.

Systems Engineer

- functional Ancient/Black/Meteor integration;
- Ice-type config plumbing;
- thaw-duration behavior;
- server reward secrecy;
- technical pool integration.

Day 13 — Warmth / Hearth Progression

Systems Engineer

- 10 Hearth levels;
- unlimited MaxWarmth verification;
- progression config plumbing;
- carry-drain configuration support.

Design / Content Lead

- costs/rates acceptance;
- Recommended Warmth values;
- region signage;
- progression feel;

- Warmth/Speed interaction playtest.

Day 14 — Economy + Collection + Feature Lock

Systems Engineer

- Pen income;
- Sell paths;
- Inventory invariants;
- Frozen Archive state;
- discovery persistence;
- economy wiring;
- Pen/thaw logical separation.

Design / Content Lead

- economy feel review;
- collection UX acceptance;
- discovery behavior acceptance;
- final launch-scope review.

Shared: L-D14-T08 - FEATURE LOCK

After Feature Lock:

No new launch gameplay systems.

9. Parallel Creature Production Track

After the Week-1 gate, creature production becomes an explicit parallel track.

While the Systems Engineer works primarily on Days 8-10, the Design / Content Lead continues:

- remaining five launch creatures;
- cleanup of existing creatures;

- silhouette mapping;
- BaseVisualScale decisions;
- content mapping;
- visual polish;
- simple reusable animation setup where useful.

Day 16 is an integration/finalization deadline, not the first day creature production begins.

10. Week 3 Ownership — Presentation, QA, Launch

Day 15 — World Art Direction

Design / Content Lead — Primary

- Norse-inspired frozen-fantasy pass;
- region differentiation;
- distant aspiration;
- Pen presentation;
- Hearth presentation;
- simple readable geometry.

Systems Engineer — Support

- integration bugs;
- profiling;
- technical cleanup;
- performance checks;
- preparation for mobile/multiplayer QA.

Day 16 — Creature Finalization + Integration

Design / Content Lead

- final creature look;

- silhouette;
- proportions;
- readability;
- visual distinction;
- asset simplification.

Systems Engineer

- Root / PrimaryPart contract;
- standardized orientation;
- runtime compatibility;
- follower compatibility;
- Pen compatibility;
- integration regressions.

Day 17 — Hero Moments

Design / Content Lead — Creative Owner

1. Great Frost
2. Impossible Ice
3. Barely-Made-It-Home
4. Legendary Smash
5. Endgame Player

Systems Engineer — Support

- gameplay hooks;
- camera/event triggers;
- state correctness;
- performance;
- technical defects.

Day 18 — UI + Onboarding

Design / Content Lead — UX Owner

- Warmth UI;
- training communication;
- countdown presentation;
- thaw-state readability;
- Recommended Warmth communication;
- first-time guidance;
- onboarding-Ice readability;
- early aspirational content.

Systems Engineer — Support

- state/event hooks;
- multiplayer-safe UI data;
- authoritative onboarding state;
- bug fixes.

Day 19 — Mobile + Multiplayer QA

Systems Engineer — Technical QA Lead

- mobile interactions;
- four-player contention;
- simultaneous reveals;
- simultaneous Great Frost;
- Special Ice competition;
- base isolation;
- visitor WarmZone;
- depleted-field onboarding;
- Special-Ice failure lifecycle.

Design / Content Lead

- mobile usability;
- UI readability;
- multiplayer feel;
- onboarding clarity;
- gameplay regressions.

Day 20 — Persistence + Balance + Exploit QA

Systems Engineer

- rejoin state;
- offline thaw accuracy;
- no offline Great Frost bonus;
- duplicate reward protection;
- save/load stress;
- inventory/Pen/Equip invariants;
- server remote abuse testing.

Design / Content Lead

- Warmth pacing;
- Speed feel;
- thaw timing;
- Special-Ice cadence.

Day 21 — Final Regression + Launch Candidate

Shared: run complete New Player, Returning Player, Midgame, Endgame, Multiplayer, Great Frost, onboarding edge-case, and Special-Ice failure journeys.

Systems Engineer — Technical Sign-Off

- saves;
- join/leave;
- mobile;

- multiplayer;
- exploit-critical systems;
- no launch-blocking server/data regression.

Design / Content Lead — Product Sign-Off

- all five Hero Moments;
- onboarding;
- progression communication;
- core USP;
- player desire;
- launch presentation quality.

Launch Candidate is declared only after both sign-offs.

11. Conflict-Risk Ownership Map

Area	Default Owner	Parallel Work Rule
<code>DataService</code>	Engineer	Design Lead does not edit during persistence work
persistent schema	Engineer	Changes require both developers to acknowledge
<code>BaseService</code>	Engineer	Base geometry may be edited separately if hierarchy contract is preserved
<code>WorldCycleService</code>	Engineer	Presentation hooks may be separate; cycle truth stays engineering-owned
<code>PlayerStateService</code>	Engineer	Single writer during migration/invariant tasks
<code>IceService</code> / <code>ThawService</code>	Engineer	Design Lead supplies authored points/config decisions
<code>CreatureService</code>	Engineer	Design Lead owns creature content/assets
world geometry	Design Lead	Engineer should not restructure without coordination
UI / presentation	Design Lead	Engineer supplies authoritative state/events
creature assets	Design Lead	Engineer owns runtime contract QA

Area	Default Owner	Parallel Work Rule
config values	Design Lead approval	One branch edits the same config record at a time
remotes/security	Engineer	No client-owned gameplay truth

12. Task Tracker Template

ID	Task	Owner	Status	Branch	Depends On	Shared Files	Runtime Tested	Reviewer	Merge
D7-R04	Hearth Prominence	Project Owner	MERGED	completed	D7-R03	presentation/world	Yes	—	Complete
D7-R05	Expanded Regions	Project Owner	MERGED	completed	D7-R04	world/regions	Yes	—	Complete
D7-R06	Recommended Warmth Signs	Project Owner	MERGED	completed	D7-R05	region signage/config	Yes	—	Complete
D7-R07	Regional Cold Severity	Project Owner	MERGED	completed	D7-R05/R06	WarmthService, RegionConfig	Yes	—	Complete
D7-R08	Ancient Ice	Engineer	NOT STARTED		D7-R07	IceService, IceConfig, authored points	No	Project Owner	
D7-R09	Black Ice	Engineer	BLOCKED		D7-R08 stable	IceService, IceConfig	No	Project Owner	
D7-R10	Meteor Ice	Engineer	BLOCKED / OPTIONAL		D7-R09 stable + time	IceService, IceConfig	No	Project Owner	
D7-R11	11-Creature Pool Expansion	Project Owner / Shared	PREP		stable Ice types + approved mapping	CreatureConfig, IceConfig/pools	No	Engineer	
D7-R12	Low-Warmth Feedback	Engineer	NOT STARTED		D7-R07	Warmth state/UI hooks	No	Project Owner	
D7-R13	Progression Validation Gate	Project Owner	BLOCKED		R08-R12 integration	full progression loop	No	Engineer	

Recommended statuses:

NOT STARTED
BLOCKED
IN PROGRESS
READY FOR REVIEW
RUNTIME TEST
READY TO MERGE
MERGED
FAILED / REWORK

13. Dependency / Blocking Protocol

If a developer is blocked:

1. mark the task `BLOCKED` ;
2. identify the exact dependency;
3. switch to a safe independent task;
4. return once the dependency merges.

Do not create speculative replacement architecture.

14. Design Ambiguity Protocol

When the engineer encounters an unspecified question:

- **Category A — Technical only:** proceed with the smallest robust implementation.
- **Category B — Existing config value:** use current canonical config.
- **Category C — Player-facing behavior:** stop and ask.
- **Category D — New system or scope expansion:** do not implement.

Examples requiring explicit approval:

- new currencies;
- new progression tracks;
- new inventory systems;

- new combat progression;
 - new pet stats;
 - new penalties;
 - new hard area gates;
 - new convenience shortcuts;
 - new monetization;
 - new Special-Ice guarantees;
 - new creature abilities.
-

15. Review Checklist

Code / Architecture

- ☐ Server owns gameplay truth.
- ☐ Remote input is validated.
- ☐ No client-owned currency/inventory/progression.
- ☐ Tunable values live in config.
- ☐ No duplicate source of truth.
- ☐ No accidental new framework.
- ☐ No unrelated refactor.
- ☐ Persistent data cannot be overwritten after failed load.
- ☐ Copy-specific creature state remains GUID-based.

☐

Hidden rewards remain hidden until reveal.

Design / Scope

☐

Exact task was implemented.

☐

No new gameplay was invented.

☐

Recommended Warmth remains guidance.

☐

MaxWarmth remains uncapped.

☐

Retrieval risk remains central.

☐

Combat/survival does not become the objective without canonical revision.

☐

Thaw completion still requires `CRACK -> CRACK -> SMASH`.

☐

Reward loop still points toward another expedition.

Runtime

☐

Core success path works.

☐

Failure path works.

☐

Nearby systems still work.

☐

Multiplayer ownership is safe if applicable.

☐

Rejoin/save behavior is tested if applicable.

☐

Temporary debug output is removed.

☐

`main` is playable after merge.

16. Daily / Session Coordination

At the start of a work session:

STARTING:
Task:
Branch:
Expected files:
Dependency:

Before touching a shared/high-conflict file:

LOCK:
Task:
File/system:

At completion:

READY:
Task:
Branch:
Runtime result:
Known issues:
Ready for review: Yes

After merge:

```
MERGED:
Task:
Commit/PR:
main is clean: Yes
Next dependency unblocked:
```

17. Production Priorities If Schedule Slips

Protect:

- Warmth;
- Warmth training;
- Hearth upgrades;
- creature Speed;
- one-Ice retrieval;
- carrying risk;
- Pen thawing;
- offline thawing;
- CRACK -> CRACK -> SMASH ;
- Great Frost reset;
- Thaw Surge;
- Special Ice;
- persistence;
- multiplayer authority;
- five-Ice progression;
- 16-creature launch target.

Simplify first:

- environmental decoration;
- unique Hearth art per level;
- Pen roaming;

- follower polish;
- complex Ice destruction;
- advanced particles;
- fancy Archive presentation;
- secondary UI animation;
- animation complexity.

If a protected item becomes impossible, stop and make an explicit design decision.

18. Summary Ownership Model

DESIGN / CONTENT LEAD

Design authority
World / layout
Content mapping
Creature production
Presentation
UI / onboarding
Economy tuning
Hero Moments
Playtest gates
Launch product sign-off

SYSTEMS ENGINEERING LEAD

Server architecture
Persistence
Data migration
Multiplayer
Bases
Remote security
World-cycle synchronization
Special-Ice lifecycle
State invariants
Integration
Exploit QA
Launch technical sign-off

SHARED

Runtime testing
Review
Regression
Feature lock
Week-1 gate
Day-21 launch decision

19. Immediate Action From Current Checkpoint

Completed:

D7-R01
D7-R02
D7-R03
D7-R04
D7-R05
D7-R06
D7-R07

Current next task:

D7-R08 - Ancient Ice

Recommended split now:

Systems Engineer:
D7-R08 -> D7-R09 -> D7-R10 if stable/time permits
D7-R12

Design / Content Lead in parallel:
prepare and approve D7-R11 creature-pool mapping
review Ice presentation / authored spawn decisions
prepare D7-R13 progression validation test
continue safe creature/content production that does not conflict with active systems work

Integration:
D7-R11
-> regression sanity pass
-> D7-R13
-> End-of-Week-1 Gate

After the Week-1 gate passes:

Systems Engineer:

Day 8 persistence

-> Day 9 multiplayer/bases

-> Day 10 production world-cycle/Special Ice

Design / Content Lead in parallel:

remaining creature production

-> content mapping

-> spawn/world preparation

-> presentation work

The immediate objective is no longer to establish the big-number region framework; that work is complete through D7-R07. The team should now finish the deeper functional Ice/content layer and validate the complete Week-1 progression loop.

Production Principle

The second engineer exists to remove technical bottlenecks, not to create a second source of game design.

The project remains strongest when:

- one person owns player-facing intent;
- one person owns core technical reliability;
- tasks have one primary owner;
- dependencies are explicit;
- `main` stays playable;
- design ambiguity is surfaced instead of guessed;
- both developers converge at validation gates.